

harbor

☐ Network policy

Tasks

Task Structure

Copy Markdown

Creating and running tasks for agentic environments

The Harbor task format is designed to be maximally flexible while still being intuitive to implement. The differences between the Harbor task format and the Terminal-Bench task format are [documented here](#).

Agent skill available

Want your coding agent to guide you through task creation? Install the `create-task` skill:

```
npx skills add harbor-framework/harbor --skill create-task
```



Creating a task

To create a task, you can use the following command:

```
harbor init --task "<org>/<name>"
```



This will create a new task directory with the following structure:

instruction.md

task.toml

environment

Dockerfile

...



You can then populate the files with your task's content.

Multi-step tasks

Harbor also supports tasks split into sequential steps, each with its own instruction, tests, and optional pre-agent setup. See [Multi-step tasks](#).

Resource requirements

Tasks can declare CPU, memory, storage, GPU, and TPU needs under `[environment]`. Harbor applies them differently per provider using enforcement policies. See [Managing Resources](#).

To evaluate an agent on your task, you can use the following command:

```
harbor run -p "<path/to/task>" -a "<agent>" -m "<model>"
```



Task files

Instruction

The instruction is a markdown file that contains the task's instruction.

Configuration & Metadata

The `task.toml` file contains the task's configuration and metadata. Metadata is arbitrary and can consist of any information a task developer wants. Config params are nested into their respective sections rather than flat.

An example is shown below:

[solution]

env = { API_KEY = "sk-test-123" }

[environment]

network_mode = "no-network" # baseline; defaults to "public" when omitted

build_timeout_sec = 600.0

docker_image = "some-org/some-name:some-tag"

os = "linux" # or "windows" to target Windows containers

cpus = 1

memory_mb = 2048

storage_mb = 10240

gpus = 0

gpu_types = ["H100", "A100"]

env = { SOME_ENV_VAR = "\${SOME_ENV_VAR}" } # harbor run requests approval from the user

[environment.tpu]

type = "v6e" # optional; omit the table if you don't need TPUs

topology = "2x4" # alias (v3, v4, v5e, v5p, v6e, v7, trillium, ironwo

A task allocates one TPU slice per pod; specify a single spec rather than a list.

Currently only the GKE environment honors this field.

[[environment.mcp_servers]]

name = "mcp-server"

transport = "streamable-http"

url = "http://mcp-server:8000/mcp"

[environment.healthcheck]

command = "curl -f http://localhost:8080/health"

interval_sec = 5.0

timeout_sec = 30.0

retries = 3

Network policy

Network access uses **baselines** (set at env start, restored between phases), **phase overrides** (optional; only during `agent.run()` or `verifier.verify()`), and **run-time merges** (`--allow-environment-host` , `--allow-agent-host`). Multi-step tasks: `[steps.*]` overrides task-level fields.

Field	Layer	Applied
<code>[environment].network_mode</code>	Baseline	Agent env start; shared verifier baseline
<code>[verifier.environment].network_mode</code>	Baseline	Separate verifier env start
<code>[steps.verifier.environment].network_mode</code>	Baseline	Per-step separate verifier env start

Field	Layer	Applied
<code>[agent].network_mode</code> , <code>[steps.agent].network_mode</code>	Override	During matching <code>agent.run()</code>
<code>[verifier].network_mode</code> , <code>[steps.verifier].network_mode</code>	Override	During matching <code>verify()</code>
<code>--allow-environment-host</code>	Run-time	Merged into <code>environment.extra_allowed_hosts</code> → <code>[environment] baseline</code>
<code>--allow-agent-host</code>	Run-time	Merged into <code>agent.extra_allowed_hosts</code> → agent phase allowlist

Verifier baseline: **shared** → `[environment]`; **separate** → `[verifier.environment]` if set, else a copy of `[environment]`.

`[environment].network_mode` defaults to `"public"`. `[agent]` / `[verifier]` (and step equivalents) are optional overrides applied only when set **and** different from the phase baseline; matching the baseline is a no-op. Modes: `public`, `no-network`, or `allowlist` with optional `allowed_hosts` (exact hostnames, IPv4/IPv6 address literals or CIDR ranges, or leading wildcard hostnames such as `*.example.com`, when supported by the selected environment; not URLs, bracketed IPv6 addresses, ports, or paths). In `allowlist` mode, empty or omitted `allowed_hosts` denies all egress. Hostnames are exact: for example, `ubuntu.com` does not allow `ask.ubuntu.com`; use a leading wildcard pattern such as `*.ubuntu.com` to allow subdomains. Wildcard apex behavior is provider-dependent. For portable tasks, include both `example.com` and `*.example.com` when both apex and subdomains are needed. Legacy `allow_internet = false` on a baseline section maps to `no-network`.

If a phase override differs from its baseline, the provider must support `dynamic_network_policy` or Harbor rejects the task. Use `verifier.environment_mode = "separate"` for a different verifier baseline without runtime switching. Pass `--allow-environment-host` for deps needed at env start; `--allow-agent-host` for deps needed only during `agent.run()` (e.g. `pypi.org`). On a `public` baseline, run-time host flags emit a warning and are ignored.

Examples: `examples/tasks/network-policy-matrix/`.

The configuration parameters are shown below:

Prop	Type	
<code>schema_version?</code>	string	▼
<code>multi_step_reward_strategy?</code>	"mean" "final" null	▼
<code>task.name?</code>	string	▼
<code>task.version?</code>	string null	▼
<code>task.description?</code>	string	▼
<code>task.authors?</code>	list[Author]	▼
<code>task.keywords?</code>	list[string]	▼
<code>metadata?</code>	object	▼
<code>verifier.timeout_sec?</code>	number	▼
<code>verifier.network_mode?</code>	"no-network" "public" "allowlist" null	▼
<code>verifier.allowed_hosts?</code>	list[string] null	▼
<code>verifier.env?</code>	object	▼
<code>verifier.user?</code>	string int null	▼
<code>verifier.environment_mode?</code>	"shared" "separate" null	▼
<code>verifier.environment?</code>	EnvironmentConfig null	▼
<code>verifier.environment.network_mode?</code>	"no-network" "public" "allowlist" null	▼
<code>verifier.environment.allowed_hosts?</code>	list[string] null	▼
<code>agent.timeout_sec?</code>	number null	▼
<code>agent.network_mode?</code>	"no-network" "public" "allowlist" null	▼
<code>agent.allowed_hosts?</code>	list[string] null	▼
<code>agent.user?</code>	string int null	▼
<code>solution.env?</code>	object	▼

environment.build_timeout_sec? number

environment.network_mode? "no-network" | "public" | "allowlist" | null

environment.allowed_hosts? list[string] | null

environment.docker_image? string | null

environment.os? "linux" | "windows"

environment.cpus? integer | null

environment.memory_mb? integer | null

environment.storage_mb? integer | null

environment.gpus? integer | null

environment.gpu_types? list[string] | null

environment.tpu? TpuSpec | null

environment.tpu.type? string

environment.tpu.topology? string

environment.allow_internet? boolean | null

environment.env? object

environment.mcp_servers? list[MCPServerConfig]

environment.skills_dir? string | null

environment.healthcheck.command? string

environment.healthcheck.interval_sec? number

environment.healthcheck.timeout_sec? number

environment.healthcheck.start_period_sec? number

environment.healthcheck.start_interval_sec? number

environment.healthcheck.retries? integer

source? string | null

During task creation, you can pass a `--metadata-template` flag with a path to a TOML file to pre-populate `task.toml` with metadata fields and config defaults:

```
harbor tasks init "<task-name>" --metadata-template task-template.toml
```

Sections in the template override Harbor's built-in defaults. Anything not specified falls back to the defaults listed above.

Environment

The environment definition is placed in an `environment/` folder. Harbor does not require any specific file to exist in that directory. Which file is required depends on the environment type being used for the evaluation. For example, to use `--env docker`, the `DockerEnvironment` class accepts any of: `[environment].docker_image`, an `environment/Dockerfile`, or `environment/docker-compose.yaml`. Setting `docker_image` lets you omit the Dockerfile when using a pre-built image. If you omit both `environment/Dockerfile` and `environment/docker-compose.yaml`, any other files in `environment/` are uploaded into the container workdir when the environment starts. Use `--force-build` only when you have a Dockerfile and want to rebuild from source instead of pulling the pre-built image. Different environment types could require other files to be present (e.g. an Apptainer environment could check for an `image.def` file). Most cloud sandbox providers only support `Dockerfile` defined environments and not docker compose.

The target container OS is declared via `[environment].os` in `task.toml`. It defaults to `"linux"`; set it to `"windows"` to target Windows containers (see [Windows tasks](#) for details). Container-side paths, file transfer, command execution, and script discovery all adapt to this value automatically.

There are a few special paths in the environment's filesystem (Linux paths shown; Windows containers use the `C:` drive equivalents — `C:/logs`, `C:/tests`, `C:/solution`):

Path	Description
<code>/logs/verifier/</code>	Contains the reward file and other verifier logs.
<code>/logs/agent/</code>	A directory agents can use to store logs from their runs.

Path	Description
<code>/solution/</code>	The solution folder is copied here by the Oracle agent at runtime and executed from the working directory.
<code>/tests/</code>	The tests folder is copied here by the Harbor harness at runtime and executed from the working directory.

The `/logs/` directory is downloaded to the host after the agent/verifier run and are often useful for debugging and analysis.

Solution (Optional)

The solution folder must contain a `solution/solve.sh` script (or `solve.bat` for Windows tasks; the right extension is selected based on `[environment].os`). Other dependencies are allowed. This folder is copied to `/solution` by the Oracle agent at runtime and executed from the working directory.

If no solution is provided, the Oracle agent cannot be used to sanity check the task.

Tests

The tests folder must contain a `tests/test.sh` script (or `test.bat` for Windows tasks). The test script should install test dependencies and verify the agent completed the instruction. In Terminal-Bench, this was done by running a `pytest` command, but this is now left to the task developer.

Other dependencies are allowed in the `tests/` folder. This folder is copied to `/tests` by the Harbor harness at runtime and executed from the working directory. E.g. `bash /tests/test.sh` is executed from `/app` in many cases.

We recommend using absolute paths in your test script to avoid relative path issues.

Importantly, the test script must produce a reward file in the `/logs/verifier/` directory. This is the file that the verifier will read to determine if the task was successful.

There are two ways to produce a reward file:

Reward File	Format	Description
<code>/logs/verifier/reward.txt</code>	Plain text (e.g. <code>1</code>)	A plain text file containing a single integer or float value, typically <code>1</code> for success or <code>0</code> for failure.
<code>/logs/verifier/reward.json</code>	JSON (e.g. <code>{ "runtime_sec": 1.23, "accuracy": 0.95, ... }</code>)	A JSON file that can define multiple metrics as rewards, but they must be floats or integers.

You may use either `reward.txt` or `reward.json` as the output of your test script. Harbor will read `reward.json` by default and fall back to `reward.txt`.

For verifiers with multiple criteria, score aggregation, and LLM judging, see [Rewardkit](#).

Often, a reward can be determined by the exit code or a unit test command.

```
>_ tests/test.sh

#!/bin/bash

uvx pytest /tests/test.py

if [ $? -eq 0 ]; then
    echo 1 > /logs/verifier/reward.txt
else
    echo 0 > /logs/verifier/reward.txt
fi
```

Verifier environment (Shared vs Separate)

By default, the verifier runs inside the **same container as the agent** — it can see the workdir, any tools the agent installed, the agent's environment variables, etc. For tasks that need an isolated grading environment — proprietary grading code that the agent must not see, a different OS for grading, a clean image with verifier-only dependencies — declare a dedicated **separate** verifier environment.

You opt in by adding `[verifier.environment]` to `task.toml`:

```
[verifier]
environment_mode = "separate" # optional when [verifier.environment] is present

[verifier.environment]
docker_image = "my-org/grading-image:latest"
cpus = 2
memory_mb = 1024
```

The two new fields under `[verifier]`:

- `environment_mode`: "shared" (default) or "separate".
- `environment`: same schema as `[environment]` (including `network_mode` baseline). Network overrides: Network policy.

Resolution rules when fields are omitted:

<code>environment_mode</code>	<code>[verifier.environment]</code>	Result
omitted	omitted	"shared"
omitted	present	"separate"
"shared"	omitted	"shared"
"shared"	present	validation error
"separate"	omitted	"separate" (uses a fresh copy of the top-level <code>[environment]</code>)
"separate"	present	"separate" (uses the verifier-specific environment)

For **separate verifier environments**, Harbor builds the verifier image from one of the task's `tests/` directories — the step's own `steps/<name>/tests/` when it provides an OS-appropriate test script, otherwise the task's top-level `tests/`. The image must provide `/tests/test.sh` (Linux) or `/tests/test.bat` (Windows) on its own; Harbor does **not** upload `tests/` at runtime. A typical layout:

 Task directory with separate verifier env

```

my-task/
├── task.toml
├── instruction.md
├── environment/
│   └── Dockerfile          # agent's environment
├── tests/
│   ├── Dockerfile        # verifier's environment (builds /tests/test.sh into the image)
│   ├── test.sh
│   └── grader.py

```

What gets transferred from the agent env to the verifier env

When a separate verifier env runs, Harbor copies these inputs from the agent env into the verifier env at the same paths:

- `/logs/artifacts/` (the agent's "publish" directory — files the agent intentionally produced for grading).
- Every artifact listed in the task-level `artifacts =` field, the trial-level artifacts, and the current step's `artifacts =` field.

Every artifact re-materializes at its **original absolute source path** in the verifier container ("no translation"): if you declared `/app/output.json`, read it at `/app/output.json`. Harbor creates parent directories during upload, so the verifier image does not need to pre-create them.

`/logs/agent/` and `/logs/verifier/` are **not** transferred implicitly. However, if you explicitly declare them as configured artifacts, they will be transferred — this is the canonical pattern for a **trajectory-grading verifier**:

```
artifacts = ["/logs/agent/trajectory.json"]
```



That single line makes the agent's trajectory file available to the separate verifier container at the same path.

Sidecar artifacts and collect hooks

Multi-container tasks (declared via `environment/docker-compose.yaml`) often hold their score signal inside a **sidecar service**: a database the agent wrote rows into, an API server that logged the agent's requests, a load generator with in-memory counters. In separate verifier mode all

containers are torn down before verification, so that evidence must be captured first. Two mechanisms work together:

1. Sidecar artifact entries — add `service` to an artifact entry to pull a file from that compose service's filesystem instead of the agent's container:

```
artifacts = [
  "/app/output.json", # from main (the agent),
  { source = "/var/log/api/requests.log", service = "api" }, # from the api sidecar
]
```

2. Collect hooks — run a snapshot command inside a service after the agent finishes, to dump runtime state (database contents, in-memory counters) into files that artifact entries then collect:

```
[[verifier.collect]]
service = "postgres"
command = "pg_dump -U postgres app > /tmp/dump.sql"
timeout_sec = 60.0

artifacts = [{ source = "/tmp/dump.sql", service = "postgres" }]
```

The verifier reads sidecar evidence at the same original paths (`/var/log/api/requests.log`, `/tmp/dump.sql`).

Shell. Commands targeting `main` run under `bash` (the agent image is harbor-built and always provides it). Commands targeting a **sidecar** run under POSIX `sh -c`, because sidecars are arbitrary third-party images and `bash` is frequently absent from minimal ones (for example the `*-alpine` variants of `postgres`, `redis`, and `nginx`). Keep sidecar collect commands POSIX-compatible. If you need bash-specific syntax (`[[ ... ]]`, arrays, `source`, process substitution) on a sidecar whose image ships `bash` — the default `postgres`, `mysql`, `redis`, and `kafka` tags all do — invoke it explicitly, e.g. `command = "bash -c '[[ -f /data/ready ]] && pg_dump ...'";` for anything elaborate, drop a script into the image and run `bash /path/snapshot.sh` to avoid nested quoting.

Anti-cheat properties. Sidecar evidence is pulled directly from each service's filesystem — a channel the agent's container cannot write to. In separate verifier mode, Harbor stops the `main`

service *before* running sidecar collect hooks and pulling sidecar artifacts, so leftover agent processes cannot interfere. Corollaries for task authors:

- Evidence collected from **sidecars** is trustworthy as long as the agent could not gain code execution on the sidecar (network access to a service $\neq$ filesystem access). Keeping sidecars unexploitable is the task author's responsibility.
- Collect hooks targeting `main` run in an environment the agent fully controlled — treat their output with the same suspicion as any agent deliverable. They are fine for "did the agent's own service work" checks, never for tamper-sensitive signals.
- The stop-`main`-first guarantee applies to single-step trials and the **final step** of a multi-step trial. Earlier steps keep the `main` container running (later steps need it), so their sidecar evidence is collected with the agent's container still live and any processes it left behind able to reach the sidecar over the network. Put tamper-sensitive sidecar evidence on the final step (or in a single-step separate-verifier task); for intermediate steps, treat sidecar evidence as agent-influenceable.

Harbor validates artifact sets at task load. Because all services share one flat `artifacts/` base dir, entries from different services whose source paths are equal or nested would collide on the same host path; Harbor emits a load-time warning and, at collection time, keeps the first claimant and skips the rest (recorded in `manifest.json`). Avoid overlapping sidecar sources: on collision only the first-collected service's content survives, so an unintended overlap can silently drop the evidence you meant to score. The one hard error is a sidecar entry whose source is not an absolute path.

Sidecar artifacts and collect hooks require a compose-capable environment provider (docker, daytona, modal, ec2, islo, gke, novita, langsmith, blaxel, beam, vercel). See

[examples/tasks/sidecar-artifacts](#) for a complete working task.

Per-step verifier environments (multi-step tasks)

Step names are unique, portable directory names directly under `steps/`, not paths. See [Multi-step tasks](#) for naming rules.

Each step can override the trial-level verifier mode under `[steps.verifier]`. Mixed shared/separate is supported — for example, an early "build" step that uses the agent env for fast feedback, plus a final "grade" step that uses a separate locked-down grading image:

 task.toml (multi-step mixed mode) 

```
[[steps]]
name = "build"
# Inherits the trial-level mode (shared by default).

[[steps]]
name = "grade"
[steps.verifier.environment]
docker_image = "my-org/grading-image:latest"
```

Step-level resolution:

- `[steps.verifier].environment_mode` (when set) wins.
- `[steps.verifier.environment]` present + mode omitted → implies `"separate"`.
- Otherwise the step inherits the trial-level resolution.

Multi-step network fields follow the same baseline/override rules; see [Network policy](#).

Tests for each step are validated against the OS of that step's *effective* verifier environment, not always the top-level `[environment].os`. So a Linux agent can be graded by a Windows verifier env (and vice versa) — Harbor checks that the corresponding `test.bat` / `test.sh` exists in the step's `tests/` dir at task-load time.

< Registry Credentials

Store credentials for pulling private task images

Publishing a task >

Publish tasks on the Harbor registry to share wit...